

Pwn Challenge Source Code

從常見弱點程式碼理解二進位漏洞觀念

一、什麼是 Pwn

在 CTF（Capture The Flag）競賽中，Pwn（發音類似 "pone"，源自駭客俚語 "own"）是指針對二進位程式（binary）進行漏洞分析與利用的題型。與 Web 題不同，Pwn 題目通常提供一支編譯好的執行檔（有時附上原始碼），選手需要找出程式中的記憶體安全性漏洞，理解其成因，並設計方法讓程式偏離原本的執行邏輯。

本教材的定位

這份教材聚焦在「閱讀原始碼、辨識漏洞類型、理解成因與防禦」，屬於觀念與程式碼審查層級的內容，**不包含**完整可運作的攻擊payload、shellcode 撰寫或逐步攻擊指令。實際的 exploit 撰寫建議在合法授權的靶場（如 pwnable.kr、pwn.college）中，搭配課程講師逐步練習。

二、C 語言記憶體模型回顧

要理解 Pwn 題目，必須先熟悉程式在記憶體中的配置方式。一個典型的行程（process）記憶體配置大致包含：

區塊	說明
Stack	存放區域變數、函式呼叫的返回位址、暫存的參數
Heap	透過 malloc / free 等動態配置的記憶體
BSS / Data	存放全域變數與靜態變數
Text	存放編譯後的機器碼（程式邏輯本身）

C 語言不像現代高階語言那樣自動做邊界檢查（bounds checking），這也是絕大多數 Pwn 漏洞的根本成因：**程式相信了「輸入資料的長度不會超過預期」，但實際上並沒有真正檢查。**

三、常見漏洞類型與示意程式碼

3.1 堆疊緩衝區溢位（Stack Buffer Overflow）

最經典的 Pwn 漏洞類型。當程式把使用者輸入寫入一個固定大小的陣列，卻沒有限制寫入長度時，多餘的資料會覆蓋到緩衝區之後的記憶體，可能包含其他區域變數，甚至函式的返回位址。

```
void vulnerable_function() {
    char buffer[64];
    // gets() 不會檢查輸入長度，是典型的危險函式
    gets(buffer);
    printf("你輸入的是: %s\n", buffer);
}
```

成因分析

`gets()` 完全不檢查輸入長度，只要輸入超過 64 bytes，就會持續往緩衝區之後的記憶體寫入。如果溢位的資料涵蓋到函式的返回位址（return address），理論上可以改變程式結束這個函式後「該跳去哪裡繼續執行」——這正是堆疊溢位危險的根本原因。

防禦觀念

- 使用有邊界檢查的函式，如 `fgets()` 取代 `gets()`。
- 編譯器層級防護：Stack Canary（在返回位址前放一個隨機值，函式返回前檢查是否被竄改）。
- 作業系統層級防護：ASLR（位址空間隨機化）、NX/DEP（讓堆疊不可執行）。

3.2 格式化字串漏洞（Format String Vulnerability）

```
void print_input(char *user_input) {
    // 危險：直接把使用者輸入當成格式化字串
    printf(user_input);
}
```

成因分析

正確寫法應該是 `printf("%s", user_input)`。上面的錯誤寫法讓使用者輸入的內容本身被當成格式化字串解析，如果輸入中包含 `%x`、`%n` 等格式化符號，攻擊者可以窺探堆疊上的資料，甚至（在較舊或防護較弱的環境）寫入任意記憶體位置。

防禦觀念

永遠明確指定格式化字串，絕不要把「使用者可控的內容」直接當成 `printf` 系列函式的第一個參數。現代編譯器通常也會針對這類寫法發出警告，應視為必須修正的錯誤，而非忽略的警告。

3.3 堆積使用釋放後漏洞（Use-After-Free）

```
struct Note *note = malloc(sizeof(struct Note));
fill_note(note);
free(note);
// 危險：free 之後指標未被清空，仍被使用
print_note(note);
```

成因分析

記憶體被 `free()` 釋放後，該區塊可能被重新配置給其他用途，但程式中的指標（`note`）並沒有被清空，仍指向那塊已經「不屬於它」的記憶體。若攻擊者能夠設法讓這塊記憶體被重新配置為攻擊者可控的內容，就可能讓程式讀取或寫入到非預期的資料。

防禦觀念

- 釋放後立即將指標設為 `NULL`（俗稱 "free 後歸零"）。
- 使用具備自動記憶體管理的語言，或啟用現代 heap allocator 的防護機制（如 tcache 相關的完整性檢查）。
- 使用靜態分析工具、AddressSanitizer 等工具在開發階段就抓出這類問題。

3.4 整數溢位（Integer Overflow）

```
void copy_data(char *input, unsigned int len) {
    char buffer[256];
    if (len < 256) {
        memcpy(buffer, input, len);
    }
}
```

成因分析

乍看之下有做長度檢查，但若 `len` 的型別是有號整數而在其他地方被設為負數，經過型別轉換成無號整數後可能變成一個非常大的數字，繞過檢查邏輯；或是計算長度時發生整數溢位（例如 `len + 1` 超過型別上限而變回一個很小的數字），都可能導致邊界檢查形同虛設。

四、程式碼審查時該問自己的問題

- 這裡的輸入長度有沒有上限？上限是誰決定的（使用者可控，還是程式固定）？
- 這個緩衝區的大小，跟實際寫入的資料大小，是同一個人在維護、同步更新的嗎？
- 指標在 `free()` 之後，後面的程式碼還有沒有機會用到它？
- 整數運算有沒有可能溢位？型別轉換（有號／無號）有沒有陷阱？
- 錯誤處理路徑（error handling）是否也一樣做了邊界檢查，還是只有正常路徑有檢查？

五、現代二進位保護機制總覽

機制	作用
Stack Canary	偵測堆疊緩衝區溢位是否覆蓋到返回位址
ASLR	每次執行時隨機化記憶體位址，增加攻擊者預測位址的難度
NX / DEP	讓資料所在的記憶體區段不可被當作程式碼執行
PIE	讓執行檔本身的載入位址也隨機化，而不只是函式庫
RELRO	限制某些重要資料結構（如 GOT）在程式執行後被竄改

這些機制彼此獨立、互相補強：單一機制被繞過不代表整個系統就不安全，這也是為什麼現代 Pwn 題目往往需要組合多種技巧、繞過多層防護才能成功，難度遠高於單純的緩衝區溢位年代。

六、學習建議

- 先扎實學好 C 語言與組合語言（x86-64 為主）基礎，理解函式呼叫時堆疊的變化。
- 熟悉 GDB、pwndbg / GEF 等除錯工具，養成「先動態觀察、再推論成因」的習慣。
- 從合法且專為教學設計的平台開始練習（如 pwnable.kr、pwn.college、picoCTF），並優先選擇有完整 writeup 教學的題目。
- 把每一次解題後的「成因」與「防禦方式」寫下來，比單純「解出來」更有長期價值。

重要提醒

對非自己擁有、未取得授權的系統或程式進行攻擊測試可能觸犯法律。本教材內容僅供理解漏洞成因與防禦觀念之用，請於合法授權之 CTF 平台或靶場中練習。